TCP part 4: how connections end

FIN · RST · TIME_WAIT

Watch the 8-min video

In one line: a connection closes in both directions with FIN and ACK; the side that closes first waits in TIME_WAIT, 60 seconds on Linux; a reset ends a connection at once, and many short connections can use up all your client ports.

WORDS YOU'LL MEET TODAY

FIN: finish: I'm done sending

TIME_WAIT: the wait, after closing, before a connection is really gone (60 s on Linux)

Ephemeral port: the temporary port a client picks for each connection

Connection pool: a few open connections reused for many requests

RST: reset: stop this connection right now

CLOSE_WAIT: the other side said goodbye, and our app hasn't closed yet

Half-close: one direction closed while the other can still send

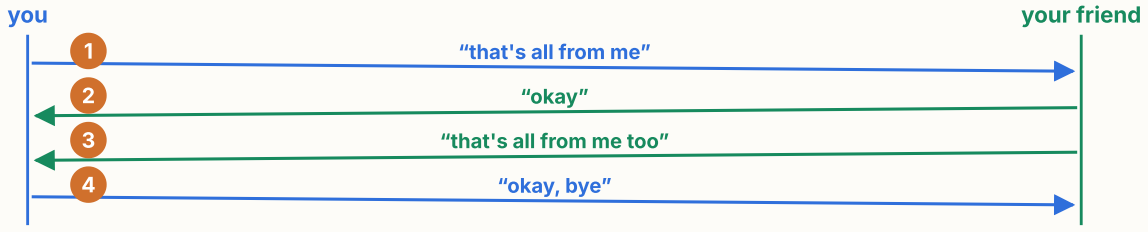
Keep-alive: keeping a connection open for the next request

1 The question: why did it run out of ports?

A busy service calls another service thousands of times a second. Suddenly every new call fails with *cannot assign requested address*. The CPU is fine, the memory is fine: it has run out of ports. To understand why, we need the end of a TCP connection, not the start.

2 An everyday picture first: ending a phone call

Ending a call politely: you say, that's all from me. Your friend says, okay. Then your friend says, that's all from me too, and you say, okay, bye. Each person says they're done and the other confirms: four messages, because there are two directions, just like the handshake in lesson 9. The rude way: you just hang up, and anything your friend was still saying is lost.



Each side says it's done and the other confirms: 4 messages, 2 directions. The rude way: just hang up (a reset).

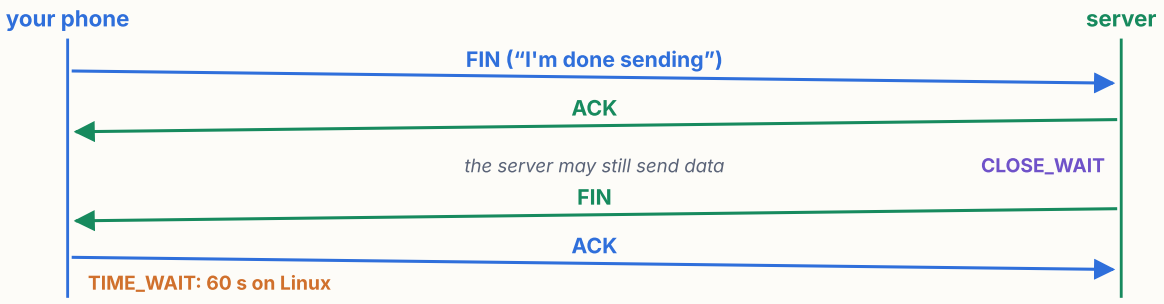
Figure 1. The analogy: ending a call politely takes four messages, because each person says they're done and the other confirms; hanging up is the reset.

3 The polite close: FIN and ACK, both ways

Your phone is done sending, so it sends a segment with the **FIN** flag. The server answers ACK, and one direction is closed. But the server may still have data to send. When it's done, it sends its own FIN, and your phone answers ACK. Both directions are closed. Often the server's ACK and FIN travel together, so you see three segments, not four.

Between the two halves the server is in **CLOSE_WAIT**: it got the goodbye and is waiting for its own app to close. If the app never closes, the connection stays there forever; thousands of them mean a bug in your

code, someone forgot to close.

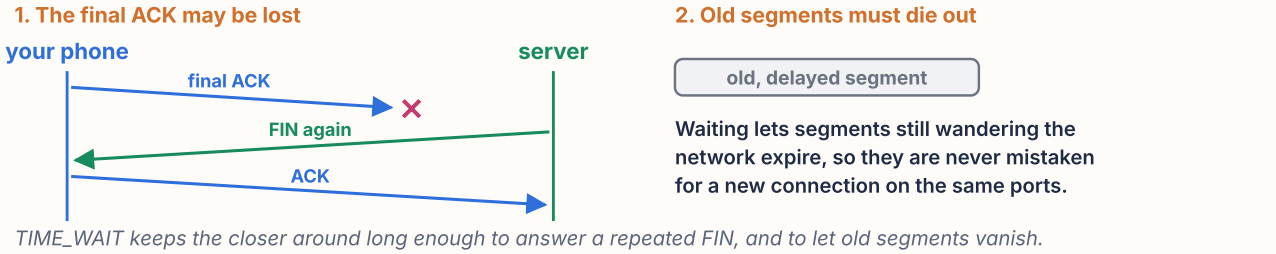


Each direction closes on its own. The server's ACK and FIN often travel together: 3 segments, not 4.

Figure 2. The polite close: FIN and ACK in each direction. The server sits in CLOSE_WAIT until its app closes; the side that closed first waits in TIME_WAIT.

4 TIME_WAIT: why the closer waits

The side that closed first, your phone, doesn't forget the connection right away. It enters **TIME_WAIT**; on Linux it waits 60 seconds. Two reasons. One: the final ACK might be lost; then the server sends its FIN again, and your phone must still be there to answer. Two: old, delayed segments from this connection might still be wandering the network; waiting lets them die out, so they are never mistaken for a new connection on the same ports.

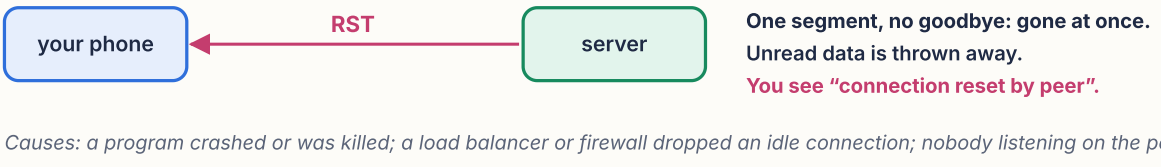


TIME_WAIT keeps the closer around long enough to answer a repeated FIN, and to let old segments vanish.

Figure 3. Why TIME_WAIT exists: a lost final ACK makes the server repeat its FIN, and the closer must still answer; and delayed old segments need time to die out.

5 The rude way: RST

A **reset** is one segment, no goodbye, and the connection is gone at once. Any data not yet read is thrown away, and you see *connection reset by peer*. Resets happen when a program crashes or is killed, when a load balancer or firewall drops an idle connection, or (lesson 9) when nobody listens on that port.



Causes: a program crashed or was killed; a load balancer or firewall dropped an idle connection; nobody listening on the port.

Figure 4. The rude way: a reset (RST) ends the connection at once and throws away unread data; you see "connection reset by peer".

6 Running out of ports

From lesson 4: a connection is named by four values, client address, client port, server address and server port. When your service calls one database at one address and one port, three of the four never change; only the client port can. So each connection to that database needs its own client port.



Calls to one database (one address, one port): each connection needs its own client port.

Figure 5. A connection is named by four values. To one database address and port, only the client port changes, so each connection needs its own.

Real numbers. Linux hands out client ports from 32,768 to 60,999: about 28,000. Every connection your service closes keeps its port busy, in TIME_WAIT, for 60 seconds. So at most about 28,000 ports every 60 seconds: roughly **470 new connections per second** to that one address and port. Open and close faster than that, and the ports run out.

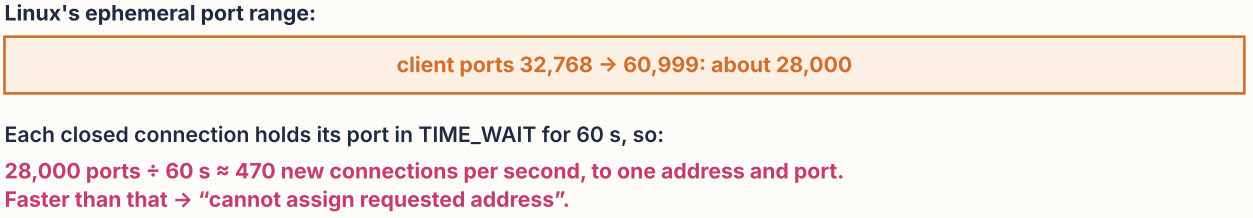


Figure 6. Running out of ports: about 28,000 client ports, each held for 60 seconds in TIME_WAIT, allow roughly 470 new connections per second to one server address and port.

The fixes, best first. Reuse connections: open a few and send many requests over each, with keep-alive and connection pools (lesson 29). Widen the port range. Let Linux safely reuse ports in TIME_WAIT for new outgoing connections, with `tcp_tw_reuse`. Spread the calls over more server addresses.

You see	It means	Do this
many TIME_WAIT on a client	too many short connections	pool them
many CLOSE_WAIT	your app forgets to close	fix the code
connection reset by peer	the other side crashed, timed out an idle connection, or refused it	check the other side

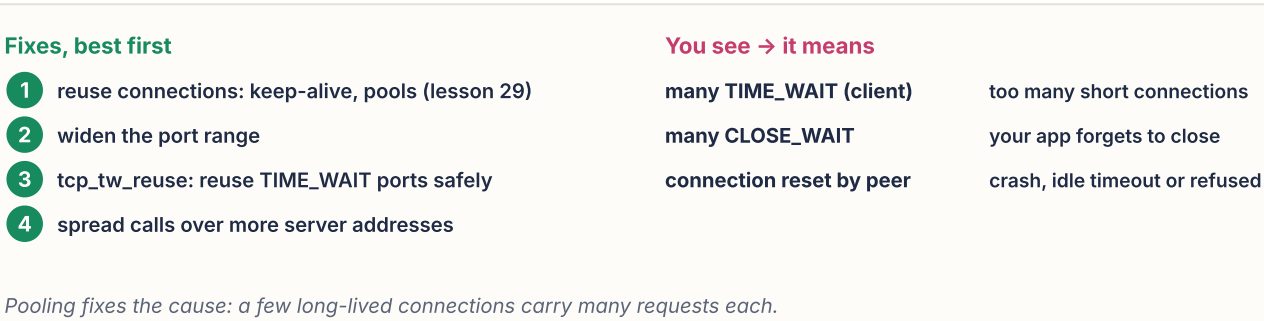


Figure 7. The fixes, best first, and a symptom checklist: TIME_WAIT piles mean short connections, CLOSE_WAIT piles mean an app that forgets to close.

PAUSE AND THINK

A service opens a new connection to its database for every query and closes it right after. It runs 1,000 queries per second; the port range is 28,000 and TIME_WAIT lasts 60 seconds. Does it run out of ports? Cover the answer and decide.

Answer. Yes. 1,000 a second, each holding a port for 60 seconds, needs 60,000 ports, and it only has 28,000. After about 28 seconds new connections start to fail. The fix: a connection pool with, say, 20 connections that stay open.

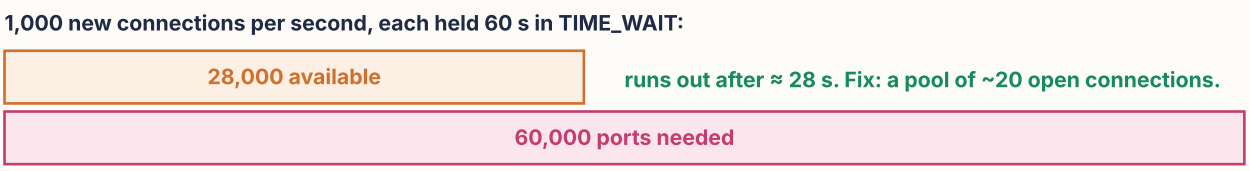


Figure 8. The pause-and-think answer: 1,000 connections a second for 60 seconds needs 60,000 ports against about 28,000, so new connections fail after about 28 seconds; a pool fixes it.

THREE COMMON MISTAKES

- **"Closing takes one message."** No. Each direction closes on its own: FIN and ACK, both ways.
- **"TIME_WAIT is a bug, so turn it off."** No. It protects you from lost ACKs and old segments. Reuse connections instead.
- **"A reset means an attack."** Usually not. Most resets are crashed programs, idle timeouts or closed ports.

RECAP IN THREE LINES

1. A connection closes in both directions: FIN, ACK, FIN, ACK.
2. The side that closes first waits in TIME_WAIT, for 60 seconds on Linux.
3. A reset ends a connection at once, and short connections can use up all your ports.

7 Interview questions: what to say in 30 seconds

Q1. How does a TCP connection close? BASIC

Each direction closes separately. The side that's done sends a FIN and the other ACKs it; later the other side sends its FIN and gets an ACK. That's four segments, often three because the second ACK and FIN travel together. The side that closed first then waits in TIME_WAIT.

Q2. What is TIME_WAIT and why does it exist? BASIC

The state the side that closed first stays in after the final ACK, 60 seconds on Linux (two maximum segment lifetimes in the standard). It lets the closer answer a repeated FIN if its last ACK was lost, and lets old delayed segments expire so they can't be mistaken for a new connection with the same addresses and ports.

Q3. A service starts failing with "cannot assign requested address". What's going on? SCENARIO

It's out of ephemeral ports. Each new connection to the same server address and port needs a distinct client port, and closed connections hold theirs in TIME_WAIT for 60 s: about 28,000 ports means roughly 470 new connections per second. Fix the cause with keep-alive and connection pools; widening the range, tcp_tw_reuse and more destination addresses help too.

Q4. You see thousands of connections in CLOSE_WAIT. What does it tell you? SCENARIO

The peer has closed, but your application never closed its side: a leak in your code, such as a missing close in an error path or a connection not returned to the pool. The kernel can't fix it; the app must close the socket.

Q5. What's the difference between FIN and RST? TRICKY

FIN is a graceful close of one direction: data already sent is delivered and acknowledged, and the other side can keep sending. RST aborts the whole connection immediately, discards data not yet read, and the other side sees “connection reset by peer”. Crashes, idle timeouts on load balancers and connections to a closed port cause resets.

Q6. Should you turn off TIME_WAIT to fix port exhaustion? TRICKY

No. It protects against lost ACKs and stray old segments. Reuse connections so you open fewer; on Linux, tcp_tw_reuse safely reuses TIME_WAIT ports for new outgoing connections using timestamps. The old tcp_tw_recycle was unsafe behind NAT and was removed.

Read more. RFC 9293 (TCP: closing, TIME_WAIT, RST), RFC 6056 (port randomisation and ephemeral ports), RFC 7323 (timestamps, used by tcp_tw_reuse), Linux ip-sysctl.txt (ip_local_port_range, tcp_tw_reuse) and tcp(7). The full written lesson is at backendroadmap.com.

Next: SD-N-012 · UDP vs TCP: when speed beats reliability backendroadmap.com